

Docker Compose - Resumo de Estudos (Projeto .NET + SQL Server)

O que é Docker Compose?

O Docker Compose é uma ferramenta que permite definir e orquestrar múltiplos containers rodando juntos como se fossem uma única aplicação.

Em vez de executar vários comandos manuais separados no terminal, o Compose usa um único arquivo YAML (`docker-compose.yml`) para configurar a infraestrutura inteira.

- Seu principal objetivo é facilitar a criação de ambientes complexos (ex: Backend + Banco de Dados).
 - Ele cria automaticamente uma rede virtual interna (Network) para que os containers possam se comunicar pelo nome do serviço.
-

Por que criar um Dockerfile e depois um Docker Compose?

- **Dockerfile:** É um arquivo de texto que contém todas as instruções para construir uma imagem. No nosso caso, ele é a "receita" apenas da nossa aplicação .NET, ensinando o Docker a compilar o código C# e gerar o pacote pronto.
- **Docker Compose:** É o "maestro" da orquestração. Ele chama o `Dockerfile` para gerar a imagem da aplicação .NET, depois vai na internet baixar a imagem pronta do SQL Server e, por fim, liga os dois na mesma rede. Sem o Compose, você teria que abrir vários terminais e rodar comandos `docker run` complexos para cada peça do sistema e tentar conectá-las na mão.

Fluxo de funcionamento da Arquitetura

1. O Compose lê o arquivo `docker-compose.yml`.
 2. Baixa a imagem do banco de dados (SQL Server).
 3. Lê o `Dockerfile` da aplicação e constrói a imagem local.
 4. Cria uma Network privada.
 5. Inicia o Banco de Dados.
 6. Espera o banco estar 100% pronto (Healthcheck).
 7. Inicia a Aplicação Web, injetando a string de conexão nela.
-

Explicação do código `docker-compose.yml`

O arquivo é dividido em serviços (`services`), que representam os containers que farão parte do sistema.

1ª Parte: Ferramenta e Infraestrutura (SQL Server)

Esta etapa configura o servidor de banco de dados que dará suporte à aplicação.

- `image:`
`[mcr.microsoft.com/mssql/server:2022-latest](https://mcr.microsoft.com/mssql/server:2022-latest)`: Define qual imagem será utilizada como base. Puxa a versão oficial do SQL Server direto da Microsoft.
- `user: root`: Garante que o container tenha permissão máxima de escrita para salvar os dados no Windows sem tomar bloqueio.
- `ports: - "1433:1433"`: Mapeia a porta do computador (Windows) para a porta interna do container. Permite conectar o VS Code ao banco.
- `environment::` Define variáveis de ambiente, como a aceitação dos termos de uso e a criação da senha do usuário administrador (`sa`).
- `volumes:` Conecta o diretório interno do container a um volume permanente. Mesmo que um container seja apagado, os dados armazenados em um volume continuam existindo.

- `healthcheck::` Um teste automatizado. O Docker fica enviando o comando `SELECT 1` para o banco de dados até ele responder, garantindo que ele está totalmente ligado.

2ª Parte: Integração com a Arquitetura (.NET)

Esta etapa constrói a sua aplicação e a conecta ao banco de dados.

- `build: context: . dockerfile: Dockerfile`: Em vez de baixar uma imagem pronta, avisa ao Docker para ler o `Dockerfile` local e construir a aplicação na hora.
 - `ports: - "5000:8080"`: Mapeia a porta 5000 do navegador para a porta 8080 do servidor C# (Kestrel) no container.
 - `environment::` Injeta configurações no código C#.
 - A `ConnectionStrings__DefaultConnection` passa o usuário, a senha e o nome do servidor (`Server=sqlserver_db`).
 - O Entity Framework lê isso e cria as tabelas automaticamente.
 - `depends_on: sqlserver: condition: service_healthy`: Cria uma trava de segurança. A aplicação .NET só liga *depois* que o SQL Server passar no teste do `healthcheck`.
-

Novos Comandos Docker utilizados

Subir a infraestrutura em segundo plano

`docker compose up -d`

Lê o arquivo `docker-compose.yml` e inicia todos os containers listados. O `-d` (detached) libera o terminal para você continuar usando.

Subir a infraestrutura reconstruindo o código

`docker compose up -d --build`

Força o Docker a ler o seu `Dockerfile` de novo e recompilar a sua aplicação C#. Obrigatório sempre que você alterar alguma coisa no código-fonte.

Derrubar a infraestrutura

`docker compose down`

Para e remove os containers e a rede virtual criados pelo Compose, mas **mantém** os dados salvos nos volumes.

Derrubar a infraestrutura e destruir os dados

```
docker compose down -v
```

Para os containers e **apaga** os volumes. Útil quando o banco de dados corrompe ou você quer limpar todas as tabelas e começar do zero.

Entrar no terminal do container (Acesso ao Banco)

```
docker exec -it sqlserver_db /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P "Senha"
```

Abre o terminal executando uma ferramenta instalada lá dentro (neste caso, o terminal SQL).

Vantagens dessa Arquitetura

- **Máquina Limpa:** Você tem um servidor backend robusto e um banco de dados relacional rodando sem instalar o SQL Server no computador físico.
- **Sincronia:** O Entity Framework Core gerencia as migrações e o banco de forma isolada através da rede virtual.
- **Reprodutibilidade:** Se o projeto for enviado para um colega de faculdade, ele só precisa rodar `docker compose up -d` e terá exatamente o mesmo ambiente em minutos.